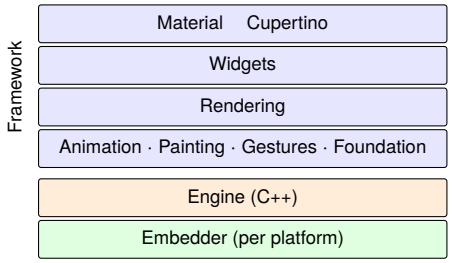
Flutter architecture and widget types

Examples 13 to 15 of `flutter_examples`

The three layers

Each layer relies only on the one below it.



Layer	Role
Framework	Dart: widgets, layout, animation, gestures
Engine	draws scenes, runs Dart, text layout, file and network I/O, accessibility
Embedder	starts the app on one OS; gives a surface, input events, life cycle

The four components of the lecture

Flutter Engine	the Engine layer: portable C++ runtime, plugins
Foundation library	bottom of the Framework: basic Dart classes
Widgets	Widgets + Rendering layers: composition of the UI
Design-specific widgets	the Material and Cupertino libraries

Not Skia any more: the default renderer is Impeller (iOS, Android API 29+, desktop). Only the web still uses Skia. Material and Cupertino work on *every* platform: Flutter draws each control itself.

The widget tree

- A widget is an **immutable description** of part of the screen.
- Widgets nest in each other from the root: the **widget tree**.
- `build` returns the widgets that describe a widget. Keep it **fast** and **without side effects**: Flutter may call it at any time.
- On a change, Flutter builds the new tree, **compares** it with the old one and redraws only what differs. You never edit the screen yourself.
- Layout rule: **constraints go down, sizes go up**.

Print the real tree

```
FloatingActionButton(
  onPressed: () => debugDumpApp(),
  child: const Icon(Icons.account_tree),
)
```

The tree appears in the terminal or VS Code's **Debug Console**; each widget is one level deeper than its parent.

```
MyApp
 MaterialApp
  ... Theme, Directionality, Navigator
   HomePage
    Scaffold
     ... Center > Column > Text, Text
     ... AppBar > Text
     ... FloatingActionButton > Icon
```

A dozen widgets in our code become more than 200 in the dump: `MaterialApp` and `Scaffold` create the others.

Two families of widgets

Family	Members
Visible	`Text`, buttons, `Image`, `Icon`: show content or take input
Layout	`Column`, `Row`, `Center`, `Padding`, `Stack`, `Scaffold`: place and size others

- `Stack`: children on top of each other, first one at the bottom.
- No new before a widget: Dart does not need it.

Gestures

`GestureDetector` is *invisible*: it makes its child react to touches. The child sets the look and the size of the touch area.

```
GestureDetector(
  onTap: () => setState(() => _taps++),
  onLongPress: () =>
      setState(() => _taps = 0),
  child: Container(width: 200,
    height: 200,
    child: Text('Taps: $_taps')),
)
```

`onTap`	a tap
`onDoubleTap`	two quick taps
`onLongPress`	press and hold
`onPanUpdate`	a drag
`onScaleUpdate`	a pinch (scale)

How a stateful widget is built

```
class TapCounterBox extends StatefulWidget {
  const TapCounterBox({super.key});
  @override
  State<TapCounterBox> createState() =>
      _TapCounterBoxState();
}

class _TapCounterBoxState
    extends State<TapCounterBox> {
  int _taps = 0;            // the state
```

Copyrighted content Dr Mohamad AOUDE

```
  @override
  Widget build(BuildContext context) =>
      Text('Taps: $_taps');
}
```

- **Two classes**: the widget (immutable, only `createState`) and its `State` (the data and `build`).
- `setState(() { ...})` runs the change, then Flutter calls `build` again; only what changed is redrawn.
- Changing a field *without* `setState`: the screen does not update.

Stateless or stateful?

Ask: *does this widget have data that changes while the app runs?*

No → stateless	`Text`, `Icon`, `Container`
Yes → stateful	`Checkbox`, `Radio`, `Slider`, `Form`, `TextField`

`Row`, `Column`, `Center`, `Padding` are layout widgets with no state of their own.

Data shared down the tree

An `InheritedWidget` passes data to every widget below it. That is how `Theme.of(context)` and `MediaQuery.of(context)` work.

The three examples

`13_widget_types`	visible and layout widgets in one page
`14_gestures`	tap $= +1$, long press $=$ reset
`15_widget_tree_dump`	press the button, read the tree